EdgeAI-Lab

Benchmarking accuracy, inference latency, and model size for efficient vision deployment

Research methodology | Version 0.1 | November 2026

Status: research framework implemented and offline-tested. No real PyTorch/ONNX/INT8 comparisons have been recorded in this report. Actual results must come from an executed benchmark using a trained model and labeled held-out data.

Research question

When a fixed trained image classifier is converted from PyTorch FP32 to ONNX Runtime FP32 and to statically quantized ONNX INT8, how do prediction agreement, accuracy, latency, and model-file size change on the same test images?

Experimental conditions

Configuration	Inference framework	Device and purpose
A. Baseline	PyTorch FP32	CPU, reference accuracy and timing
B. Export	ONNX Runtime FP32	CPU, export parity and runtime effect
C. Quantized	ONNX Runtime QDQ INT8	CPU, static calibrated quantization
D. Optional	PyTorch CUDA	GPU, separate hardware comparison

Core methodological controls

Use one checkpoint and verify checkpoint/ONNX hashes; the same image preprocessing, class indices, and held-out evaluation images for every configuration. Run CPU experiments at batch 1 and 8, with repeated trials after warmup. Execute each backend in a separate process; use equal thread counts and record the system environment.

Model origin and dataset separation

Either import an existing compatible trained torchvision state_dict whose data provenance and preprocessing are known, or train MobileNetV3-small from random initialization on Imagenette training images. Select the best checkpoint using a development subset from train only. Keep the official val folder unseen until final evaluation. Calibrate static INT8 only with images from train.

Evaluation protocol

Measurements

Measure	Operational definition
Top-1 / Top-5 accuracy	Fraction of held-out images whose target label is in top-1 / top-5 logits
Prediction agreement	Fraction of samples with matching top-1 class to PyTorch FP32
Latency	Median and 95th percentile of repeated model-forward timings
Throughput	Input images divided by average measured forward time
Model size	On-disk bytes of model checkpoint and ONNX artifacts
Process memory	Sampled peak resident set size for the entire isolated worker; not isolated model RAM or GPU VRAM

Quantization details

Apply ONNX Runtime static post-training quantization to Conv/MatMul/Gemm operators using min/max calibration and QDQ representation. Calibration samples must be from the training split. The inference session must load and execute the quantized model before any benchmark claims. Quantized accuracy can decrease; INT8 can be slower on unsupported CPUs.

Benchmark integrity

CPU models are compared on one CPU. CUDA measurements are reported in a separate group, not as an equivalent backend speedup. Input preprocessing is done before the timed forward pass. GPU timing includes synchronization and device transfer in the implementation. Benchmark reports record CPU thread counts, batch sizes, run sample sizes, warmups, and variability across timing repeats.

Interpretation and limitations

A smaller model file does not guarantee lower latency, and whole-process RSS should not be interpreted as model-only memory. The evaluation images may be insufficiently representative for deployment. Results on Imagenette are not the official ImageNet-1K validation score. Performance on one machine cannot establish universal gains.

Evidence still required

Complete a real training or checkpoint-adoption run, verify ONNX numerical parity, calibrate the INT8 model, execute both batch sizes on one labeled holdout set, inspect misclassifications and quantization-induced changes, then include measured tables and figures in a revised research report.

References

ONNX Runtime quantization documentation; PyTorch ONNX export tutorial; torchvision MobileNetV3 documentation; fastai Imagenette dataset documentation. Full links are listed in the repository README.